

Wdrożenie HES-Lite v1.0 do FairWear

Podsumowanie wykonawcze

Najbezpieczniejszy kierunek implementacji jest taki: **HES-Lite pozostaje całkowicie deterministyczną logiką w `common`, UI w `entry` wyłącznie konsumuje wynik, a `EngineFairWearService` jest adapterem pomiędzy istniejącym silnikiem FairWear, HES-em, warstwą podpisu i ViewModelami.** Nie przenosiłbym żadnej logiki punktowej do `entry` ani `watch`.

To dobrze pasuje do HarmonyOS/ArkTS: Huawei opisuje ArkTS jako język zachowujący podstawową składnię TypeScript, ale wzmacniający statyczne sprawdzanie i analizę, więc fakt, że kod przechodzi testy Node, **nie zastępuje** kompilacji ArkTS w DevEco. ¹ Oficjalny HarmonyOS SDK udostępnia osobne warstwy m.in. ArkUI, Crypto Architecture Kit, Sensor Service Kit, Universal Keystore Kit i Wear Engine Kit, co odpowiada sensownemu podziałowi FairWear na czystą logikę, UI, sensory i kryptografię. ²

Najważniejsza decyzja HES pozostaje taka:

$$HES = 15\% CRF + 15\% RHR + 15\% MVPA + 15\% SleepRegularity + 12.5\% Steps + 12.5\% SleepDuration$$

Pięć obowiązkowych komponentów Core daje **70% modelu**:

$$RHR + MVPA + SleepRegularity + Steps + SleepDuration = 70\%$$

VO₂max, HRV i HRR **nie są ozdobnikami** — kiedy są dostępne, faktycznie zmieniają HES. Kiedy ich brakuje, nie są zgadywane i zmniejszają `Evidence Coverage`.

Finalna semantyka:

Score
= quality of the observed profile under the prototype HES rules

Evidence Coverage
= how much of the eight-component model is actually observed

Data Confidence
= how complete the available longitudinal evidence is

Missing evidence
!= bad evidence

Live sensor value
!= longitudinal HES input

Wszystkie dokładne wagi, krzywe, progi A/B/C, minima liczby dni/nocy i progi Confidence należy traktować jako **prototype product assumptions, not clinically validated**. HES nie powinien być opisywany jako AI ani jako predykcja zdrowia.

Przygotowałem również jeden połączony plik silnika:

[Pobierz HesEngine.ets](#)

Plik ma być docelowo umieszczony pod:

```
common/src/main/ets/hes/HesEngine.ets
```

Nie twierdzą, że ten konkretny plik przeszedł DevEco: w tej sesji nie mam dostępu do repozytorium ani `fairwear-ui.zip`, więc nie mogłem wykonać kompilacji ArkTS ani Waszych 66 testów. Kod jest implementacyjnym wariantem specyfikacji, który trzeba zderzyć z istniejącymi `HesTypes.ets` i eksportami z paczki.

Główne ryzyka i założenia

Największym blockerem nie jest HES, tylko brak dostępu w tej rozmowie do czterech plików, które określają dokładną integrację: `types.ets`, `defaults.ets`, `ViewModels.ets` i `common/oh-package.json5`. Dlatego **nazwa HAR, dokładne pola MonthlyReport / TierClaim, dokładne pola MonthVM / ClaimVM oraz obecne źródło tieru są UNSPECIFIED**.

Nie wolno zgadywać tych nazw i potem „dopasowywać kompilatora”.

Drugi blocker to decyzja architektoniczna:

Czy istniejący tier generowany przez obecny silnik, np. `RiskScorer`, zostaje źródłem tieru, czy tier A/B/C z HES-Lite ma go zastąpić?

Tego nie powinien rozstrzygać kod. To musi zatwierdzić człowiek przed `feature/engine-service`.

`types.ets` i `defaults.ets` pozostają nietknięte. W kroku dotyczącym K4 dwie zmiany wymagają jawnej zgody człowieka; do tego momentu można napisać failing tests, ale nie zmieniać defaults.

Architektura i finalny silnik HES-Lite

Komponenty i wagi

To powinno być jedno źródło prawdy. Nie trzymałbym drugiej kopii wag w UI, mapperze ani serwisie.

```
export enum HesComponentId {  
    CRF = 'crf',  
    RHR = 'rhr',  
    MVPA = 'mvpa',  
}
```

```

    SLEEP_REGULARITY = 'sleepRegularity',
    STEPS = 'steps',
    SLEEP_DURATION = 'sleepDuration',
    HRV = 'hrv',
    HRR = 'hrr'
}

export interface ComponentDefinition {
  id: HesComponentId;
  label: string;
  baseWeight: number;
  requiredCore: boolean;
  minimumCount: number;
  targetCount: number;
  curve: CurvePoint[];
}

export const COMPONENTS: ComponentDefinition[] = [
  {
    id: HesComponentId.CRF,
    label: 'VO2max',
    baseWeight: 0.15,
    requiredCore: false,
    minimumCount: 1,
    targetCount: 2,
    curve: CRF_CURVE
  },
  {
    id: HesComponentId.RHR,
    label: 'Resting heart rate',
    baseWeight: 0.15,
    requiredCore: true,
    minimumCount: 10,
    targetCount: 14,
    curve: RHR_CURVE
  },
  {
    id: HesComponentId.MVPA,
    label: 'Active minutes',
    baseWeight: 0.15,
    requiredCore: true,
    minimumCount: 14,
    targetCount: 21,
    curve: MVPA_CURVE
  },
  {
    id: HesComponentId.SLEEP_REGULARITY,
    label: 'Sleep timing regularity',
    baseWeight: 0.15,
    requiredCore: true,
    minimumCount: 14,

```

```

    targetCount: 21,
    curve: SLEEP_REGULARITY_CURVE
  },
  {
    id: HesComponentId.STEPS,
    label: 'Daily steps',
    baseWeight: 0.125,
    requiredCore: true,
    minimumCount: 14,
    targetCount: 21,
    curve: STEPS_CURVE
  },
  {
    id: HesComponentId.SLEEP_DURATION,
    label: 'Sleep duration',
    baseWeight: 0.125,
    requiredCore: true,
    minimumCount: 14,
    targetCount: 21,
    curve: SLEEP_DURATION_CURVE
  },
  {
    id: HesComponentId.HRV,
    label: 'Heart rate variability',
    baseWeight: 0.10,
    requiredCore: false,
    minimumCount: 7,
    targetCount: 14,
    curve: HRV_CURVE
  },
  {
    id: HesComponentId.HRR,
    label: 'Heart rate recovery',
    baseWeight: 0.05,
    requiredCore: false,
    minimumCount: 2,
    targetCount: 3,
    curve: HRR_CURVE
  }
];

```

Kontrola konfiguracji powinna być częścią testów:

```

export function assertHesConfiguration(): void {
  let totalWeight: number = 0;

  for (let i: number = 0; i < COMPONENTS.length; i++) {
    totalWeight += COMPONENTS[i].baseWeight;
  }
}

```

```

    if (Math.abs(totalWeight - 1.0) > 0.000001) {
      throw new Error('HES component weights must sum to 1.0.');
```

Krzywe scoringowe

Finalne punkty kontrolne:

```

export const CRF_CURVE: CurvePoint[] = [
  { x: 5, y: 10 },
  { x: 10, y: 20 },
  { x: 25, y: 45 },
  { x: 50, y: 70 },
  { x: 75, y: 88 },
  { x: 90, y: 97 },
  { x: 95, y: 100 }
];

export const RHR_CURVE: CurvePoint[] = [
  { x: 40, y: 90 },
  { x: 45, y: 98 },
  { x: 50, y: 100 },
  { x: 55, y: 100 },
  { x: 60, y: 95 },
  { x: 65, y: 90 },
  { x: 70, y: 84 },
  { x: 75, y: 76 },
  { x: 80, y: 66 },
  { x: 90, y: 45 },
  { x: 100, y: 20 },
  { x: 110, y: 5 }
];

export const MVPA_CURVE: CurvePoint[] = [
  { x: 0, y: 0 },
  { x: 30, y: 20 },
  { x: 60, y: 40 },
  { x: 100, y: 60 },
  { x: 150, y: 80 },
  { x: 250, y: 95 },
  { x: 300, y: 100 }
];

export const SLEEP_REGULARITY_CURVE: CurvePoint[] = [
  { x: 30, y: 100 },
  { x: 45, y: 90 },
  { x: 60, y: 80 },
  { x: 90, y: 60 },
  { x: 120, y: 40 },
```

```

    { x: 180, y: 15 },
    { x: 240, y: 0 }
  ];

export const STEPS_CURVE: CurvePoint[] = [
  { x: 1000, y: 0 },
  { x: 2000, y: 20 },
  { x: 4000, y: 50 },
  { x: 5000, y: 65 },
  { x: 6000, y: 80 },
  { x: 7000, y: 90 },
  { x: 8000, y: 95 },
  { x: 10000, y: 100 }
];

export const SLEEP_DURATION_CURVE: CurvePoint[] = [
  { x: 4.5, y: 0 },
  { x: 5.0, y: 25 },
  { x: 5.5, y: 45 },
  { x: 6.0, y: 65 },
  { x: 6.5, y: 82 },
  { x: 7.0, y: 95 },
  { x: 7.5, y: 100 },
  { x: 8.0, y: 100 },
  { x: 8.5, y: 100 },
  { x: 9.0, y: 100 },
  { x: 9.5, y: 95 },
  { x: 10.0, y: 90 },
  { x: 11.0, y: 80 },
  { x: 12.0, y: 65 }
];

export const HRV_CURVE: CurvePoint[] = [
  { x: 5, y: 10 },
  { x: 10, y: 25 },
  { x: 25, y: 50 },
  { x: 50, y: 75 },
  { x: 75, y: 90 },
  { x: 90, y: 97 },
  { x: 95, y: 100 }
];

export const HRR_CURVE: CurvePoint[] = [
  { x: 12, y: 10 },
  { x: 18, y: 30 },
  { x: 24, y: 55 },
  { x: 30, y: 70 },
  { x: 36, y: 82 },
  { x: 42, y: 92 },
  { x: 50, y: 100 }
];

```

Interpretacja wartości wejściowych musi być jednoznaczna:

HES component	Input to curve
CRF	vo2maxPer centile
RHR	median resting HR over recent completed history
MVPA	equivalent weekly minutes
Sleep regularity	TimingSD in minutes
Steps	mean completed-day steps
Sleep duration	median hours/night
HRV	reference percentile
HRR	HRR1 in bpm

VO₂max i HRV są w hackathonie percentylami dla **jednej zdefiniowanej grupy referencyjnej**. Nie należy dorabiać brakujących percentyli na podstawie innych komponentów.

Jedna funkcja interpolująca

```
export interface CurvePoint {
  x: number;
  y: number;
}

export function clamp(
  value: number,
  minValue: number,
  maxValue: number
): number {
  if (value < minValue) {
    return minValue;
  }

  if (value > maxValue) {
    return maxValue;
  }

  return value;
}

export function piecewiseScore(
  value: number,
  points: CurvePoint[]
): number {
  if (points.length < 2) {
    throw new Error('A scoring curve needs at least two points.');
```

```

if (value <= points[0].x) {
  return clamp(points[0].y, 0, 100);
}

const last: CurvePoint = points[points.length - 1];

if (value >= last.x) {
  return clamp(last.y, 0, 100);
}

for (let i: number = 0; i < points.length - 1; i++) {
  const a: CurvePoint = points[i];
  const b: CurvePoint = points[i + 1];

  if (value >= a.x && value <= b.x) {
    const span: number = b.x - a.x;

    if (span <= 0) {
      throw new Error(
        'Scoring curve x values must be strictly increasing.'
      );
    }

    const t: number = (value - a.x) / span;

    return clamp(
      a.y + t * (b.y - a.y),
      0,
      100
    );
  }
}

throw new Error('Value did not match the scoring curve.');
```

Przykład:

Steps = 3000

2000 -> 20

4000 -> 50

$t = (3000 - 2000) / (4000 - 2000)$
 $= 0.5$

$score = 20 + 0.5 * (50 - 20)$
 $= 35$

Poprawne przejście przez północ

To powinno zostać w logice i mieć osobny test, bo typowy zapis minut od północy robi z 23:30 → 00:30 różnicę 1380 minut.

```
export function minutesSince1800(
  minutesSinceMidnight: number
): number {
  const day: number = 1440;
  const anchor: number = 1080; // 18:00

  return (
    (minutesSinceMidnight - anchor) % day
    + day
  ) % day;
}
```

Czyli:

```
23:30 -> 1410 minutes from midnight
00:30 -> 30 minutes from midnight
```

Anchored:

```
23:30 -> 330
00:30 -> 390
```

Difference = 60 minutes

Regularność:

```
export function sleepTimingSd(
  bedtimeMinutesSinceMidnight: number[],
  wakeMinutesSinceMidnight: number[]
): number {
  if (
    bedtimeMinutesSinceMidnight.length === 0 ||
    wakeMinutesSinceMidnight.length === 0 ||
    bedtimeMinutesSinceMidnight.length !==
      wakeMinutesSinceMidnight.length
  ) {
    throw new Error(
      'Bedtime and wake-time arrays must be non-empty and have equal length.'
    );
  }

  const bedRelative: number[] = [];
  const wakeRelative: number[] = [];
```

```

for (
  let i: number = 0;
  i < bedtimeMinutesSinceMidnight.length;
  i++
) {
  bedRelative.push(
    minutesSince1800(
      bedtimeMinutesSinceMidnight[i]
    )
  );

  wakeRelative.push(
    minutesSince1800(
      wakeMinutesSinceMidnight[i]
    )
  );
}

const bedSd: number =
  standardDeviation(bedRelative);

const wakeSd: number =
  standardDeviation(wakeRelative);

return Math.sqrt(
  (
    bedSd * bedSd
    +
    wakeSd * wakeSd
  ) / 2
);
}

```

Nie nazywałbym tego w UI **SRI**, bo implementujecie własny **Sleep Timing Regularity**, a nie pełny algorytm Sleep Regularity Index.

Dane, Coverage, Confidence i wynik

Sufficiency

Finalne minima:

```

// CRF / V02max
minimumCount = 1;
targetCount = 2;
// window: 90 days

// RHR
minimumCount = 10;

```

```

targetCount = 14;
// main recent window: 28 completed days

// MVPA
minimumCount = 14;
targetCount = 21;

// Sleep timing regularity
minimumCount = 14;
targetCount = 21;

// Steps
minimumCount = 14;
targetCount = 21;

// Sleep duration
minimumCount = 14;
targetCount = 21;

// HRV
minimumCount = 7;
targetCount = 14;

// HRR
minimumCount = 2;
targetCount = 3;
// comparable sessions, window: 60 days

```

Jakość komponentu:

$$q_i = \min(\text{validCount}_i / \text{targetCount}_i, 1)$$

```

export function quality(
  validCount: number,
  targetCount: number
): number {
  if (targetCount <= 0) {
    throw new Error(
      'Target count must be greater than zero.'
    );
  }

  return clamp(
    validCount / targetCount,
    0,
    1
  );
}

```

Availability:

```
const available: boolean =
  observation !== undefined &&
  observation.value !== undefined &&
  validCount >= definition.minimumCount;
```

Najważniejszy warunek:

```
export function hasAllCore(
  components: ComponentResult[]
): boolean {
  for (
    let i: number = 0;
    i < components.length;
    i++
  ) {
    const component: ComponentResult =
      components[i];

    if (
      component.requiredCore &&
      !component.available
    ) {
      return false;
    }
  }

  return true;
}
```

Jeżeli choć jeden Core jest niewystarczający:

```
score      = unavailable
tier       = NONE
confidence = NOT_ENOUGH_DATA
status     = INSUFFICIENT_DATA
```

Ola **nie** dostaje Tier C.

Evidence Coverage

$$Coverage = 100 \sum_i w_i I_i$$

```
export function coverageWeight(
  components: ComponentResult[]
): number {
  let sum: number = 0;
```

```

for (
  let i: number = 0;
  i < components.length;
  i++
) {
  if (components[i].available) {
    sum += components[i].baseWeight;
  }
}

return sum;
}

export function evidenceCoverage(
  components: ComponentResult[]
): number {
  return Math.round(
    coverageWeight(components) * 100
  );
}

```

To daje dokładnie:

```

Core only
= 70%

Core + CRF
= 85%

Core + HRR
= 75%

Core + CRF + HRR
= 90%

Core + CRF + HRV
= 95%

Everything
= 100%

```

Jeżeli dostępne metryki u Oli sumują się np. do 0.425, UI pokazuje:

```

Evidence Coverage
43%

```

mimo że HES nie jest jeszcze możliwy.

Final HES

$$ObservedHES = \frac{\sum_i w_i I_i S_i}{\sum_i w_i I_i}$$

$$HES = round(ObservedHES)$$

```
export function calculateHes(
  input: HesInput
): HesResult {
  const components: ComponentResult[] =
    buildComponentResults(input);

  const denominator: number =
    coverageWeight(components);

  const coverage: number =
    Math.round(denominator * 100);

  if (!hasAllCore(components)) {
    return {
      tier: HesTier.NONE,
      coverage,
      confidence:
        HesConfidence.NOT_ENOUGH_DATA,
      status:
        HesStatus.INSUFFICIENT_DATA,
      components,
      why: buildWhyThisTier(
        components,
        false
      )
    };
  }

  if (denominator <= 0) {
    throw new Error(
      'No HES evidence is available.'
    );
  }

  let numerator: number = 0;
  let qualityNumerator: number = 0;

  for (
    let i: number = 0;
    i < components.length;
    i++
  ) {
    const component: ComponentResult =
      components[i];
```

```

    if (
      component.available &&
      component.score !== undefined
    ) {
      numerator +=
        component.baseWeight *
        component.score;

      qualityNumerator +=
        component.baseWeight *
        component.quality;

      component.effectiveWeight =
        component.baseWeight /
        denominator;
    }
  }

  const observed: number =
    numerator / denominator;

  const score: number =
    Math.round(observed);

  const weightedQuality: number =
    qualityNumerator / denominator;

  const confidenceIndex: number =
    denominator *
    weightedQuality;

  return {
    score,
    tier: tierForScore(score),
    coverage,
    confidence:
      confidenceForIndex(
        confidenceIndex
      ),
    confidenceIndex,
    weightedQuality,
    status: HesStatus.OK,
    components,
    why: buildWhyThisTier(
      components,
      true
    )
  };
}

```

To jest jawna renormalizacja tylko na dostępnych komponentach. Kluczowe jest, że nie jest ukryta:

```
HES 84
Tier A
Evidence Coverage 90%
Data Confidence High
```

a nie samo:

```
84
```

Confidence

$$WeightedQuality = \frac{\sum w_i I_i q_i}{\sum w_i I_i}$$

$$ConfidenceIndex = CoverageFraction \times WeightedQuality$$

```
export function confidenceForIndex(
  index: number
): HesConfidence {
  if (index >= 0.80) {
    return HesConfidence.HIGH;
  }

  if (index >= 0.60) {
    return HesConfidence.MEDIUM;
  }

  return HesConfidence.LOW;
}
```

Progi:

```
>= 0.80
HIGH

>= 0.60 and < 0.80
MEDIUM

< 0.60
LOW

Missing Core
NOT_ENOUGH_DATA
```

To ma ważną konsekwencję:

Core only
Coverage = 70%

Even with perfect quality:
ConfidenceIndex <= 0.70

Therefore:
Data Confidence = Medium

Czyli Kasia `70% / Medium` wynika naturalnie z modelu.

Tier

```
export function tierForScore(  
  score: number  
): HesTier {  
  if (score >= 80) {  
    return HesTier.A;  
  }  
  
  if (score >= 60) {  
    return HesTier.B;  
  }  
  
  return HesTier.C;  
}
```

A >= 80
B >= 60 and < 80
C < 60
NONE = Core incomplete

Te progi są **produktowe**, a nie kliniczne.

Why this tier, live data i testy

Why this tier

Do UI nie przekazywałbym tylko `score`. Każdy komponent powinien zachować swój wynik, wagę bazową, efektywną i availability.

```
export interface ComponentResult {  
  id: HesComponentId;  
  label: string;  
  
  rawValue?: number;  
  score?: number;
```

```

    available: boolean;
    requiredCore: boolean;

    validCount: number;
    minimumCount: number;
    targetCount: number;

    quality: number;

    baseWeight: number;
    effectiveWeight?: number;
  }

```

Efektywna waga:

$$effectiveWeight_i = \frac{baseWeight_i}{\sum availableWeights}$$

Przykład przy Coverage 90% :

$$15/90 = 16.67\%$$

To **nie zmienia bazowej wagi 15%**. Służy wyłącznie pokazaniu, jak zbudowany został aktualny wynik.

```

export interface WhyThisTier {
  strongest: ComponentResult[];

  improvementOpportunities:
    ComponentResult[];

  missingEvidence:
    ComponentResult[];

  measuredSoFar:
    ComponentResult[];

  neededForScore:
    ComponentResult[];

  scoreBreakdown:
    ScoreBreakdownRow[];
}

```

Logika:

```

export function buildWhyThisTier(
  components: ComponentResult[],
  scoreAvailable: boolean

```

```

): WhyThisTier {
  const measured: ComponentResult[] = [];
  const missingOptional: ComponentResult[] = [];
  const neededForScore: ComponentResult[] = [];
  const availableScored: ComponentResult[] = [];
  const breakdown: ScoreBreakdownRow[] = [];

  for (
    let i: number = 0;
    i < components.length;
    i++
  ) {
    const component: ComponentResult =
      components[i];

    if (component.available) {
      measured.push(component);

      if (component.score !== undefined) {
        availableScored.push(component);
      }

    } else if (component.requiredCore) {
      neededForScore.push(component);

    } else {
      missingOptional.push(component);
    }

    if (
      scoreAvailable &&
      component.available &&
      component.score !== undefined &&
      component.effectiveWeight !== undefined
    ) {
      breakdown.push({
        id: component.id,
        label: component.label,
        score: component.score,
        baseWeight:
          component.baseWeight,
        effectiveWeight:
          component.effectiveWeight
      });
    }
  }

  const strongest: ComponentResult[] =
    availableScored.slice();

```

```

strongest.sort(
  (
    a: ComponentResult,
    b: ComponentResult
  ): number =>
    componentScoreValue(b) -
    componentScoreValue(a)
);

const improvement: ComponentResult[] =
  availableScored.slice();

improvement.sort(
  (
    a: ComponentResult,
    b: ComponentResult
  ): number =>
    componentScoreValue(a) -
    componentScoreValue(b)
);

return {
  strongest:
    scoreAvailable
      ? strongest.slice(0, 3)
      : [],

  improvementOpportunities:
    scoreAvailable
      ? improvement.slice(0, 3)
      : [],

  missingEvidence:
    missingOptional,

  measuredSoFar:
    measured,

  neededForScore:
    neededForScore,

  scoreBreakdown:
    breakdown
};
}

```

Dla Tomka oczekiwane UI według przekazanej specyfikacji:

HES 79
 Tier B
 Coverage 90%
 Data Confidence High

Strongest
 Sleep timing regularity
 Daily steps
 VO2max

Improvement opportunities
 Heart rate recovery
 Active minutes
 Resting heart rate

Missing evidence
 Heart rate variability

1 point to Tier A

Dla Oli:

No score yet

Evidence Coverage
 43%

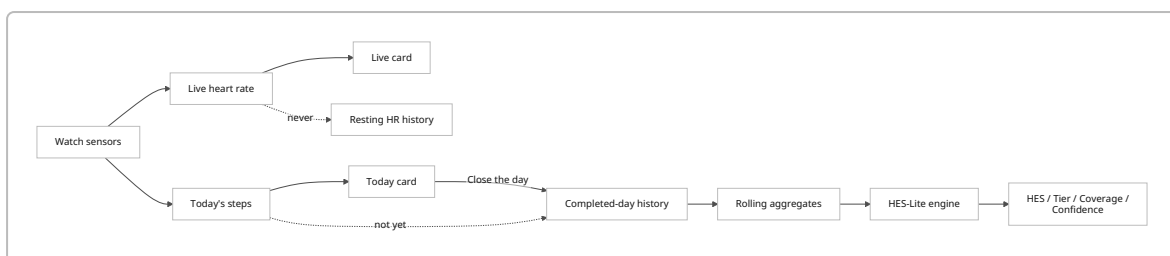
Data confidence
 Not enough data

Measured so far
 ...

Sleep timing regularity
 0 of 14 nights
 Needed for a score

Live nie jest HES

To jest jedna z najważniejszych granic całej architektury:



Zmiana:

```
Live HR  
70 -> 145
```

ma zmieniać kartę Live, ale:

```
HES 79 -> 79
```

Tak samo:

```
Today's steps  
1200 -> 8000
```

nie zmienia wyniku.

Dopiero:

```
Close the day
```

powoduje:

```
today steps  
  ->  
completed day  
  ->  
rolling history  
  ->  
new Steps component  
  ->  
HES recalculation
```

To jest bardzo dobry element demonstracyjny, bo pokazuje, że system nie udaje, iż pojedynczy skok tętą jest długoterminową oceną.

Minimalne testy HES

Paczka według opisu ma **43 testy HES i 23 testy warstwy demo/UI-service, czyli 66 łącznie**. To są wyniki oczekiwane z przekazanego zadania, a nie wyniki przeze mnie uruchomione.

Rdzeń testów powinien obejmować co najmniej:

```
describe('HES-Lite v1.0', () => {
```

```

it('scores the lower steps boundary', () => {
  expect(
    piecewiseScore(1000, STEPS_CURVE)
  ).toEqual(0);
});

it('scores an exact steps control point', () => {
  expect(
    piecewiseScore(2000, STEPS_CURVE)
  ).toEqual(20);
});

it('interpolates steps linearly', () => {
  expect(
    piecewiseScore(3000, STEPS_CURVE)
  ).toEqual(35);
});

it('scores 150 MVPA minutes as 80', () => {
  expect(
    piecewiseScore(150, MVPA_CURVE)
  ).toEqual(80);
});

it('scores resting heart rate at 55 as 100', () => {
  expect(
    piecewiseScore(55, RHR_CURVE)
  ).toEqual(100);
});

it('scores resting heart rate at 110 as 5', () => {
  expect(
    piecewiseScore(110, RHR_CURVE)
  ).toEqual(5);
});

it('handles midnight correctly', () => {
  const beforeMidnight =
    minutesSince1800(23 * 60 + 30);

  const afterMidnight =
    minutesSince1800(30);

  expect(
    Math.abs(
      afterMidnight - beforeMidnight
    )
  ).toEqual(60);
});

it('does not score missing HRV as zero', () => {

```

```

    // Core remains valid.
    // HRV must be unavailable.
    // Coverage drops by 10 percentage points.
    // HES is still available.
  });

  it('does not issue a score without all Core evidence', () => {
    // Sleep regularity below 14 nights.

    // Expected:
    // score === undefined
    // tier === NONE
    // status === INSUFFICIENT_DATA
  });

  it('gives Core-only evidence 70 percent coverage', () => {
    // Expected coverage === 70
  });

  it('gives full evidence 100 percent coverage', () => {
    // Expected coverage === 100
  });

  it('rounds the final displayed HES to an integer', () => {
    // 84.49 -> 84
    // 84.50 -> 85
  });

  it('never returns NaN for empty evidence', () => {
    // Expected:
    // INSUFFICIENT_DATA
  });

  it('does not change HES when live heart rate changes', () => {
    // HES before = 79
    // live HR 70 -> 145
    // HES after = 79
  });

  it('does not change HES when live daily steps change', () => {
    // HES before = 79
    // today steps 1200 -> 8000
    // HES after = 79
  });

  it('recalculates HES after closing a day', () => {
    // historyLength + 1
    // completed daily values enter history
    // HES is recalculated
  });

```

```
});
```

Koniecznie dodatkowo:

```
it('keeps all base weights summing to exactly 1.0', () => {
  assertHesConfiguration();
});

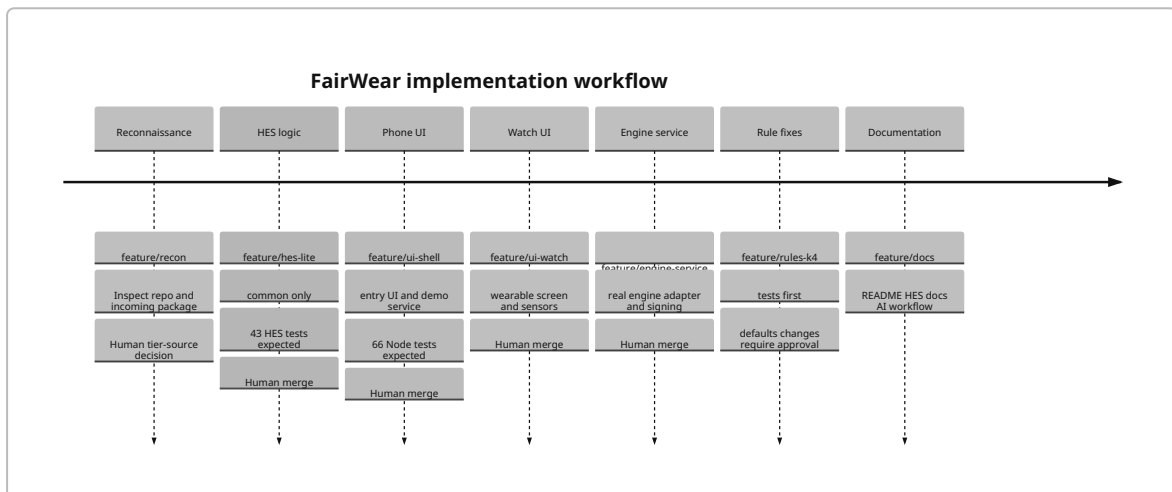
it('keeps all effective weights summing to approximately 1.0', () => {
  // Only when a HES exists.
});

it('never lists missing optional evidence as an improvement opportunity', ()
=> {
  // Missing HRV belongs under Missing evidence.
});

it('does not assign Tier C when Core evidence is incomplete', () => {
  // Expected Tier.NONE.
});
```

Integracja repozytorium i workflow

Ponieważ każdy etap ma być osobną gałęzią, a merge wykonuje człowiek, zależności powinny wyglądać tak:



Rozpoznanie

Ze względu na zasadę „każdy krok na osobnej gałęzi” użyłbym:

```
git switch main
git pull --ff-only
git switch -c feature/recon
```

Rozpakowanie:

```
mkdir -p _incoming/fairwear-ui

unzip fairwear-ui.zip \
  -d _incoming/fairwear-ui
```

Gitignore:

```
grep -qxF "_incoming/" .gitignore \
|| printf "\n_incoming/\n" >> .gitignore
```

Najpierw odszukałbym pliki, zamiast zgadywać ich ścieżki:

```
find . \
  \( -name "types.ets" \
  -o -name "defaults.ets" \
  -o -name "IMPLEMENTATION.md" \
  -o -name "ViewModels.ets" \
  -o -name "oh-package.json5" \) \
  -print
```

Potem:

```
cat _incoming/fairwear-ui/README.md

cat common/oh-package.json5
cat entry/oh-package.json5
```

Oraz znalezione:

```
types.ets
defaults.ets
IMPLEMENTATION.md
ui/model/ViewModels.ets
```

Na końcu kroku należy ustalić trzy rzeczy:

```
HAR package name
  = UNSPECIFIED until common/oh-package.json5 is inspected

Current tier source
  = UNSPECIFIED until existing engine is inspected

MonthlyReport/TierClaim field mapping
  = UNSPECIFIED until types.ets and ViewModels.ets are inspected
```

I zatrzymać się na decyzji człowieka:

```
Should HES-Lite replace the existing tier source?
YES / NO
```

Nie robiłbym automatycznej podmiany `RiskScorer`, nawet jeżeli taki symbol faktycznie istnieje.

HES-Lite

Po ludzkim merge poprzedniego kroku:

```
git switch main
git pull --ff-only
git switch -c feature/hes-lite
```

Kopie:

```
cp -R \
_incoming/fairwear-ui/common/src/main/ets/hes/ \
common/src/main/ets/

cp \
_incoming/fairwear-ui/common/src/test/Hes.test.ets \
common/src/test/
```

Tools bez nadpisywania:

```
mkdir -p tools

cp -n \
_incoming/fairwear-ui/tools/* \
tools/
```

Do:

```
common/src/test/List.test.ets
```

dodać wywołanie:

```
hesTest();
```

Eksporty z:

```
_incoming/fairwear-ui/common/Index.ets
```

należy **dopisać** do:

```
common/Index.ets
```

Jeżeli istnieje konflikt:

```
median  
mean
```

wariant bezpieczny:

```
export {  
  median as hesMedian,  
  mean as hesMean  
} from './src/main/ets/hes/HesEngine';
```

Node:

```
tools/run-logic-tests.sh \  
  common \  
  common/src/test/Hes.test.ets \  
  hes
```

Oczekiwany wynik zgodnie z paczką:

```
43/43
```

Nie nazywać tego wynikiem osiągniętym, dopóki rzeczywiście nie zostanie uruchomiony.

Build ArkTS robiłbym w DevEco jako build modułu `common`. Nie podaję wymyślonego celu `hvmigor`, ponieważ bez `build-profile.json5` i struktury projektu dokładna komenda CLI jest **UNSPECIFIED**.

ArkTS ma silniejsze statyczne sprawdzanie niż zwykły TypeScript, więc ten krok rzeczywiście może ujawnić rzeczy, których Node nie wykrył. ¹

Raport:

```
mkdir -p docs
```

Do:

```
docs/test-results.txt
```

wpisać faktycznie uruchomione:

```
HES Node tests: 43/43
DevEco Local Test: ...
Common HAR build: ...
ArkTS fixes: ...
```

UI telefonu

```
git switch main
git pull --ff-only
git switch -c feature/ui-shell
```

Jeżeli `entry` nie zależy od HAR:

```
{
  "dependencies": {
    "common": "file:../common"
  }
}
```

ale tylko wtedy, gdy nazwa HAR faktycznie jest:

```
common
```

Potem:

```
ohpm install
```

Kopie:

```
cp -R \
_incoming/fairwear-ui/entry/src/main/ets/ui/ \
entry/src/main/ets/

cp \
_incoming/fairwear-ui/entry/src/main/ets/pages/*.ets \
entry/src/main/ets/pages/

cp \
_incoming/fairwear-ui/entry/src/test/UiService.test.ets \
entry/src/test/
```

Testy całej logiki:

```
tools/run-logic-tests.sh
```

Oczekiwane:

```
HES:
43/43

UI/service:
23/23

Total:
66/66
```

Po buildzie `entry` dopiero ręcznie sprawdzić całą macierz Ania/Marek/Kasia/Tomek/Ewa/Ola i screenshoty.

Watch

```
git switch main
git pull --ff-only
git switch -c feature/ui-watch
```

Docelowe pliki:

```
watch/src/main/ets/pages/Index.ets

watch/src/main/ets/ui/DayDialog.ets

watch/src/main/ets/ui/WatchModels.ets
```

Dokładną nazwę ability trzeba odczytać z:

```
watch/src/main/module.json5
```

a nie przepisywać z przykładu.

Oficjalna platforma HarmonyOS udostępnia osobne systemowe zestawy dla sensorów i urządzeń wearable, ale dokładny permission/API używany przez projekt należy sprawdzić w deklaracjach SDK dla używanego API level. ²

Engine service

```
git switch main
git pull --ff-only
git switch -c feature/engine-service
```

Ten branch zaczyna się **dopiero po**:

```
K4-K6 green
+
human tier-source decision
```

Plik:

```
entry/src/main/ets/ui/service/EngineFairWearService.ets
```

Aktywacja:

```
FairWear.use(
    new EngineFairWearService()
);
```

w:

```
Index.aboutToAppear()
```

Warstwa kryptograficzna powinna zostać poza HES. Huawei w aktualnym SDK rozdziela Crypto Architecture Kit od Universal Keystore Kit; dokumentacja HarmonyOS zawiera również ścieżki dotyczące podpisywania i weryfikowania ECDSA. ³

Nie wpisywałbym do raportu „hardware-attested”. Wymaganie projektu mówi wyraźnie o fallbacku software na emulatorze.

Reguły K4

```
git switch main  
git pull --ff-only  
git switch -c feature/rules-k4
```

Najpierw test czerwony, potem kod.

Można bez zgody człowieka zacząć testować:

```
charging cannot hide a gap  
claim does not expose reasons  
public key stays outside token  
token <= 512 characters
```

Natomiast w chwili, gdy implementacja wymaga zmiany:

```
defaults.ets
```

STOP.

Dotyczy zwłaszcza:

```
suspicious-gap share rule starts at 2 gaps  
  
night =  
at least 4 hours worn between 00:00 and 06:00  
  
minimum heart-rate readings =  
12 in the 24-hour pre-gap window
```

Bez zgody człowieka można mieć failing test dokumentujący oczekiwanie, ale nie wolno zmienić defaultów.

Jeżeli zmiana formatu `TierClaim` wymagałaby zmiany:

```
types.ets
```

ten sam STOP.

Dokumentacja

```
git switch main
git pull --ff-only
git switch -c feature/docs
```

Docelowo:

```
README.md
docs/HES.md
AI_WORKFLOW.md
docs/test-results.txt
docs/screenshots/
```

Tekst:

All weights, curves, thresholds and minimum counts are prototype product assumptions, not clinically validated.

oraz:

HES is implemented as deterministic rules and scoring curves.
It is not a trained AI model.

Mapowanie modeli i EngineFairWearService

MonthlyReport i TierClaim

Tutaj nie będę zgadywał nazw pól. W tej rozmowie nie ma dostępnego repozytorium ani retrievable `fairwear-ui.zip`, więc **nie można uczciwie stworzyć exact field-to-field mapping**.

Tabela implementacyjna przed krokiem integracji powinna zostać wypełniona z realnych deklaracji:

Semantyka	<code>types.ets</code>	<code>ViewModels.ets</code>	Reguła
Month identifier	UNSPECIFIED	UNSPECIFIED	direct/map to display month
Wear compliance	UNSPECIFIED	UNSPECIFIED	direct percentage/value
Compliance threshold	UNSPECIFIED	UNSPECIFIED	rule configuration
Eligibility	UNSPECIFIED	UNSPECIFIED	recompute after HES availability

Semantyka	types.ets	ViewModels.ets	Reguła
Daily wear records	UNSPECIFIED	UNSPECIFIED	map to Day VM
Day state	UNSPECIFIED	UNSPECIFIED	COMPLIANT/SHORT/SUSPICIOUS/PAUSED
Existing tier	UNSPECIFIED	UNSPECIFIED	human decision required
HES score	not necessarily in MonthlyReport	UNSPECIFIED	HesMapper.apply(...)
HES Coverage	not necessarily in MonthlyReport	UNSPECIFIED	HesMapper.apply(...)
HES Confidence	not necessarily in MonthlyReport	UNSPECIFIED	HesMapper.apply(...)
Claim tier	UNSPECIFIED	UNSPECIFIED	direct display
Claim eligibility	UNSPECIFIED	UNSPECIFIED	partner learns yes/no only
Claim nonce	UNSPECIFIED	normally verification state	exact field UNSPECIFIED
HES in claim	must be absent	n/a	never serialize
Coverage in claim	must be absent	n/a	never serialize
Suspicion reason in claim	must be absent	n/a	never serialize
Public key in claim/token	must be absent	n/a	register separately
kid	token envelope, not necessarily TierClaim	verification UI	8-char hex requirement from project

Po otwarciu realnych plików ta tabela powinna zostać zamieniona na coś typu:

```
MonthlyReport.<realField>
->
MonthVM.<realField>
```

bez aliasów wymyślonych przez implementatora.

Import HAR

Aktualnie:

```
HAR package name = UNSPECIFIED
```

Trzeba odczytać:

```
common/oh-package.json5
```

Jeżeli:

```
{  
  "name": "common"  
}
```

zostaje:

```
import {  
  ...  
} from 'common';
```

Jeżeli jest np.:

```
{  
  "name": "@fairwear/common"  
}
```

zmienić importy w:

```
entry/src/main/ets/ui/service/HesMapper.ets  
  
entry/src/main/ets/ui/service/MockFairWearService.ets  
  
entry/src/main/ets/ui/service/EngineFairWearService.ets
```

na:

```
import {  
  ...  
} from '@fairwear/common';
```

Nie zmieniać nazwy HAR tylko po to, żeby pasowała do gotowego UI.

EngineFairWearService

Poniższy kod jest świadomie minimalnym pseudokodem. Exact constructor names dla Waszych `MonthVM`, `ClaimVM`, `HesSession`, `ReportBuilder`, signerów i token envelope są **UNSPECIFIED**, dopóki nie zostaną odczytane z repo.

```
// EngineFairWearService.ets

// Replace 'UNSPECIFIED_HAR_PACKAGE' with the actual package name
// from common/oh-package.json5.

import {
  HesPersonas,
  HesSession,
  ReportBuilder,
  TierClaim
} from 'UNSPECIFIED_HAR_PACKAGE';

import {
  FairWearService
} from './FairWearContract';

import {
  HesMapper
} from './HesMapper';

export class EngineFairWearService
  implements FairWearService {

  private sessions:
    Map<string, HesSession> =
      new Map<string, HesSession>();

  private usedNonces:
    Set<string> =
      new Set<string>();

  private demo: boolean = true;

  personas(): PersonaVM[] {
    const ids: string[] = [
      'ania',
      'marek',
      'kasia',
      'tomek',
      'ewa',
      'ola'
    ];
  }
}
```

```

    const result: PersonaVM[] = [];

    for (
        let i: number = 0;
        i < ids.length;
        i++
    ) {
        result.push(
            this.personaVmFor(ids[i])
        );
    }

    return result;
}

month(
    personId: string
): MonthVM {
    const session: HesSession =
        this.sessionFor(personId);

    const report: MonthlyReport =
        new ReportBuilder(
            session.history()
        ).build();

    let vm: MonthVM =
        this.mapMonthlyReport(report);

    vm = HesMapper.apply(
        vm,
        session.result(),
        session.historyLength()
    );

    vm.eligible =
        vm.wearCompliance >=
            vm.requiredCompliance
        &&
        !vm.flagged
        &&
        vm.hesAvailable;

    return vm;
}

today(
    personId: string
): DaySegment[] {

```

```

    const session: HesSession =
        this.sessionFor(personId);

    const slots: WearSlot[] =
        session.todaySlots();

    const result: DaySegment[] = [];

    for (
        let i: number = 0;
        i < slots.length;
        i++
    ) {
        result.push(
            this.toDaySegment(
                slots[i]
            )
        );
    }

    return result;
}

live(
    personId: string
): LiveVM {
    return this.sessionFor(
        personId
    ).live();
}

setLive(
    personId: string,
    live: LiveInput
): void {
    const session: HesSession =
        this.sessionFor(personId);

    session.setLive(live);

    // Live heart rate must not replace longitudinal resting HR.
    // Today's running steps must not enter the HES history yet.
}

closeDay(
    personId: string
): MonthVM {
    const session: HesSession =

```

```

        this.sessionFor(personId);

        // The session commits today's step counter
        // as a completed historical day.
        session.closeDay();

        return this.month(personId);
    }

```

```

resetDemo(
    personId?: string
): void {
    if (personId !== undefined) {
        this.sessions.delete(personId);
        this.sessionFor(personId);
        return;
    }

```

```

    this.sessions.clear();

```

```

    const ids: string[] = [
        'ania',
        'marek',
        'kasia',
        'tomek',
        'ewa',
        'ola'
    ];

```

```

    for (
        let i: number = 0;
        i < ids.length;
        i++
    ) {
        this.sessionFor(ids[i]);
    }
}

```

```

requestNonce(): string {
    // Platform implementation boundary:
    //
    // const random =
    //   cryptoFramework.createRandom();
    //
    // const bytes =
    //   random.generateRandomSync(16).data;
    //
    // return base64UrlWithoutPadding(bytes);
    //

```

```

    // 16 bytes produce a 22-character
    // unpadded base64url nonce.

    return this.platformRandomNonce16();
}

async signClaim(
  personId: string,
  nonce: string
): Promise<SignedClaim> {

  const month: MonthVM =
    this.month(personId);

  const claim: TierClaim =
    this.buildPartnerClaim(
      month,
      nonce
    );

  // Claim payload must not contain:
  //
  // HES
  // Coverage
  // component scores
  // suspicious flag
  // reason for ineligibility

  try {
    return await this.signWithHuks(
      claim
    );
  } catch (error) {

    return await this.signWithSoftwareKey(
      claim
    );
  }
}

async verifyClaim(
  token: string,
  expectedNonce: string
): Promise<VerificationResult> {

  const envelope: ClaimEnvelope =
    this.decodeEnvelope(token);

```

```

if (
  envelope.claim.nonce !==
  expectedNonce
) {
  return VerificationResult.invalid(
    'Nonce mismatch'
  );
}

if (
  this.usedNonces.has(
    envelope.claim.nonce
  )
) {
  return VerificationResult.used();
}

const publicKey: Uint8Array | undefined =
  this.partnerKeyRegistry.get(
    envelope.kid
  );

if (publicKey === undefined) {
  return VerificationResult.invalid(
    'Unknown key'
  );
}

const valid: boolean =
  await this.verifyEcdsa(
    envelope.claim,
    envelope.signature,
    publicKey
  );

if (!valid) {
  return VerificationResult.invalid(
    'Invalid signature'
  );
}

this.usedNonces.add(
  envelope.claim.nonce
);

return VerificationResult.valid(
  envelope.claim
);
}

```

```

tamper(
    token: string
): string {
    const envelope: ClaimEnvelope =
        this.decodeEnvelope(token);

    if (
        envelope.claim.tier === 'B' ||
        envelope.claim.tier === 'C'
    ) {
        envelope.claim.tier = 'A';
    } else {

        // Always modify at least one serialized byte.
        envelope.claim.nonce =
            this.changeAtLeastOneByte(
                envelope.claim.nonce
            );
    }

    // Deliberately keep the old signature.
    // Verification must therefore fail.

    return this.encodeEnvelope(
        envelope
    );
}

isDemo(): boolean {
    return this.demo;
}
}

```

Huawei udostępnia Crypto Architecture Kit i Universal Keystore Kit jako osobne systemowe komponenty SDK, a dokumentacja HarmonyOS wskazuje ECDSA signing/signature verification jako wspierany scenariusz. ³ Exact wywołania HUKS powinny zostać skompilowane przeciwko deklaracjom SDK projektu, a nie przepisywane z pamięci.

Dla `requestNonce()` użycie kryptograficznego generatora losowego jest właściwym kierunkiem; oficjalny ekosystem Huawei dokumentuje Crypto Architecture Kit, natomiast dokładną sygnaturę `createRandom()/generateRandomSync()` należy potwierdzić bezpośrednio w SDK używanym przez DevEco. ²

Minimalne testy integracyjne serwisu:

```

describe('EngineFairWearService', () => {

    it('returns Ania as Tier A and eligible', () => {

```

```

    // Expected:
    // HES 92
    // Tier A
    // Coverage 100%
    // eligible = true
  });

  it('returns Marek as flagged and not eligible', () => {
    // Expected:
    // HES 75
    // Tier B
    // Coverage 85%
    // eligible = false
  });

  it('returns Kasia as Tier B and eligible', () => {
    // Expected:
    // HES 73
    // Tier B
    // Coverage 70%
    // eligible = true
  });

  it('returns Tomek at 79 and Tier B', () => {
    // Expected:
    // HES 79
    // Tier B
    // Coverage 90%
  });

  it('returns Ewa at 59 and Tier C', () => {
    // Expected:
    // HES 59
    // Tier C
    // Coverage 75%
  });

  it('returns no score for Ola', () => {
    // Expected:
    // tier NONE
    // no HES
    // Coverage 43%
  });

  it('keeps Marek and Kasia wear compliance equal', () => {
    // Existing FairWear rule expectation.
  });

  it('does not change HES when live heart rate changes', () => {
    const before: MonthVM =
      service.month('tomek');
  });

```

```

    service.setLive(
      'tomek',
      {
        heartRate: 145
      }
    );

    const after: MonthVM =
      service.month('tomek');

    expect(
      after.hes
    ).assertEqual(
      before.hes
    );
  });

  it('does not expose private scoring evidence in the claim', () => {
    // Decode payload.
    //
    // Must not contain:
    // flag
    // susp
    // reason
    // hes
    // coverage
  });

  it('rejects a tampered token', async () => {
    // sign
    // tamper
    // verify
    // Expected invalid signature
  });

  it('rejects nonce reuse', async () => {
    // First verification succeeds.
    // Second verification returns Already used.
  });
});

```

Decyzje końcowe i granice implementacji

Najważniejsze jest to, czego **nie** należy teraz zmieniać:

```

types.ets
  DO NOT CHANGE WITHOUT HUMAN APPROVAL

```

```
defaults.ets
DO NOT CHANGE WITHOUT HUMAN APPROVAL
```

Jeżeli kod z paczki wymaga nowego typu HES, używać:

```
common/src/main/ets/hes/HesTypes.ets
```

Nie dokładać HES do globalnych typów tylko dla wygody.

Jeżeli krok K4 wymaga nowego progu w `defaults.ets`, prawidłowa sekwencja jest:

1. Write the failing test.
2. Run the test.
3. Record the failure.
4. Stop.
5. Ask the human for approval to change defaults.ets.
6. Only after approval, change the default.
7. Run the test again.

Tak samo dla `types.ets`.

Najważniejsza nierozstrzygnięta decyzja pozostaje:

```
Current FairWear tier source:
UNSPECIFIED

HES-Lite replacing it:
HUMAN DECISION REQUIRED
```

Moja rekomendacja architektoniczna jest taka, żeby **nie podmieniać tieru po cichu**. Jeżeli człowiek zdecyduje, że HES jest nowym źródłem A/B/C, wtedy `HesSession.result().tier` staje się źródłem tieru w adapterze. Jeżeli stary `RiskScorer` ma pozostać źródłem decyzji FairWear, HES powinien zostać osobnym wynikiem wellness/evidence i nie należy mieszać obu semantyk pod jednym polem `tier`.

Najbardziej sensowna finalna architektura wygląda więc tak:

```
common
|
|— existing FairWear rules
|— wear compliance
|— suspicious-gap logic
|— HES-Lite
|   |— scoring curves
|   |— sufficiency
```

```

|   |— Coverage
|   |— Confidence
|   |— Why-this-tier data
|
└─ NO UI

```

```

entry
|
|— FairWearService contract
|— MockFairWearService
|— EngineFairWearService
|— HesMapper
|— signing adapter
└─ ArkUI

```

```

watch
|
|— sensor-facing logic
|— live state
|— wear-state presentation
└─ ArkUI watch face

```

HarmonyOS oficjalnie rozdziela UI, systemowe sensory, kryptografię i keystore na osobne elementy SDK, więc taki podział jest zgodny z charakterem platformy i — co ważniejsze — z Waszą zasadą, że `common` ma pozostać czystą logiką. ²

Finalne zdanie do dokumentacji HES powinno być dokładnie ostrożne:

HES-Lite is a deterministic, evidence-informed prototype score combining longitudinal activity, sleep and physiological signals.

Missing measurements are never guessed. Evidence completeness is reported separately through Evidence Coverage and Data Confidence.

All weights, curves, thresholds and minimum counts are prototype product assumptions, not clinically validated.

HES-Lite is not a trained AI model, a diagnosis, or an individual health or disease prediction.

A finalne komunikaty, których **nie umieszczać**:

```

hardware-attested
AI detects cheating
uses real Huawei Health data
tested on Watch 6
HES predicts health

```

```
HES predicts disease
HES predicts claims
clinically validated
Tier A means healthy
Tier C means unhealthy
```

Pełny połączony silnik do dalszego zderzenia z realnymi typami repo:

Pobierz common/src/main/ets/hes/HesEngine.ets

1 2 3 HarmonyOS NEXT Develop - HUAWEI Developers

<https://developer.huawei.com/consumer/en/harmonyos/develop/>